A programozás alapjai 1 - BMEVIHIAA01 2025/26/1

Ballisztikai számítási program

Készítette: Horváth Ádám József,
M74IPN
horvadam06@gmail.com

Tartalomjegyzék

A programozás alapjai 1 - BMEVIHIAA01 2025/26/1 1
1. Specifikáció 6
1.1.0 Program célja 6
1.2.0 Főbb funkciók 6
1.3.0 Bemeneti adatok formátuma és típusai 6
1.3.1 Általános formátum 6
1.3.2 Példa bemenet 7
1.3.3 Fájltípusok 7
1.3.4 Parsing szabályok 8
1.3.2. Kapcsolatok felépítése 9
1.3.3. Számítás 9
1.3.4 Parsing szabályok 9
1.4.0 Kimeneti adatok 9
1.4.1 Alapértelmezett kimenet példa(terminal) 9
1.4.2 Mentési lehetőségek 9
1.5.0 Követelmények (funkcionális követelmények) 10
1.5.1 Automatikus ajánlás 10
1.5.2 Adatkezelés 10
1.5.3 Hibakezelés 10
1.5.4 Tesztelhetőség 10
1.6.0 Tesztelés 10
1.6.1 Interpreter teszt 11
2. Adatszerkezetek 12
2.1.0 Changelog 12
2.2.0 Adatkezelés 12
2.2.1 Láncolt strázsált lista 12
2.3.0 Löveg lista 14
2.3.1 Löveg adatszerkezet 14
2.3.2 Löveg adatszerkezet főbb függvényei 15
2.4.0 Bázisok, Célpontok 16
2.4.1 Alap pozíció adatok 16
2.4.2 Bázis lista 16
2.4.3 Célpontok 18
2.4.4 Célpontok adatszerkezetek főbb függvényei 19

Horváth Ádám József
M74IPN

2.5.0 Program flowchart 22
2.5.1 Rendszerindítás és inicializáció 23
2.5.2 Konfiguráció betöltése 23
2.5.3 Parancsértelmezés és fő vezérlési ciklus 23
2.5.4 Adatszerkezetek kezelése 23
2.5.5. Kilépés és memóriafelszabadítás 24
3. Főbb algoritmusok 24
3.1.0 Adatbeolvasás és feldolgozás algoritmusa 24
3.1.1 Tokenizálás és elemzés 24
3.1.2 Scan függvény 25
3.1.3 Tokenizálás (strtoken használatával) 26
3.2.0 Ballisztikai számítások 27
3.2.1 Koordináta transzformációk 27
3.2.2 ENU (East-North-Up) transzformáció 27
3.2.3 RK4 (Runge-Kutta 4. rendű) numerikus integráció 28
3.2.4 Ballisztikai megoldás keresése 28
3.3.0 Regionális keresési algoritmus (calculations.c) 29
3.3.1 Region definition 29
3.3.2 Region-aware keresés (kétfázisú) 29
3.3.3 Távolság számítások 30
3.4.0 Láncolt lista kezelési algoritmusok 30
3.4.1 Strázsált lista struktúra 30
3.4.2 Rendezett beszúrás (ID alapján) 31
3.4.3 Keresés optimalizált úton 31
3.4.4 Memóriafelszabadítás (mélységi bejárással) 32
3.5.0 Konfiguráció kezelés algoritmusa (load.c) 32
3.5.1 Konfig fájl generálás 32
3.5.2 Konfig fájl betöltés 32
3.6.0 Hibakezelés és naplózás algoritmusai 33
3.6.1 Hibajelentés formázás 33
3.6.2 Naplózás fájlba 33
3.6.3 Eredménymentés formázás 34
4. Felhasználói dokumentáció 34
4.1.0 Telepítés és indítás 34
4.1.1 Fordítás 34

4.1.2 ANCI színek engedélyezése (Windows) 35
4.2 Kezdeti beállítás 35
4.1.1 Első indításkor 35
4.3.0 Főmenü és alapvető működés 36
4.3.1 Indítás utáni képernyő 36
4.4.0 Parancsok részletes leírása 37
4.4.1 Adatkezelési parancsok 37
4.4.2 Számítási parancsok 38
4.4.3 Rendszerparancsok 38
4.5.0 Adatfájlok formátuma 39
4.5.1 Konfigurációs fájl (config.txt)txt 39
4.5.2 Bázis fájl (bases.txt) 39
4.5.3 Célpont fájl (targets.txt) 39
4.5.4 Löveg katalógus (ammo_catalouge.txt) 39
4.6 Kimeneti formátumok 39
4.6.1 Képernyőn megjelenő eredmény 39
4.6.2 Naplófájl (log.txt) 39
4.7.0 Regionális keresési funkció 40
4.8.0 Hibakezelés és hibajelentések 40
4.8.1 Gyakori hibák és megoldások 40
4.8.2 Hibakeresési mód 41
4.9.0 Példa teljes munkamenet 41
4.10.0 Gyorsbillentyűk és tippek 42
5. Programozói dokumentáció 42
5.1 Projekt struktúra és függőségek 42
5.1.1 Fájlszerkezet 42
5.1.2 Főbb függőségek 43
5.2.0 Főbb modulok és funkcióik 43
5.2.1 main.c - Főprogram 43
5.2.2 commands.c/h - Parancsértelmezés 43
5.2.3 load.c/h - Fájlbeolvasás és konfiguráció 44
5.2.4 storage.c/h - Adatszerkezetek 45
5.2.5 calculations.c/h - Ballisztikai számítások 46
5.2.6 debug.c/h - Tesztelés és hibakeresés 47
5.3 Adatfolyam és vezérlés 48

5.3.1 Program indítási folyamat 48
5.3.2 Számítási folyamat 49
5.4 Memóriakezelés 50
5.4.1 Dinamikus memória foglalás 50
5.4.2 Memória felszabadítás 50
5.4.3 Memóriaszivárgás ellenőrzés 50
5.5 Hibakezelés 51
5.5.1 HRESULT hibakódok 51
5.5.2 Hibakezelési stratégia 51
5.6 Optimalizációk 51
5.6.1 Regionális keresés 51
5.6.2 Lista műveletek 51
5.6.3 Numerikus számítások 51
5.7 Bővíthetőség és karbantarthatóság 52
5.7.1 Moduláris design 52
5.7.2 Konvenciók és stílus 52
6. Teszt dokumentáció 52
6.1 Tesztelési stratégia 52
6.1.1 Tesztelési szintek 52
6.1.2 Automatikus tesztek 52
6.2 Tesztesetek 52
6.2.1 Funkcionális tesztek 52
6.2.2 Határérték tesztek 54
6.3 Tesztadatok 55
6.3.1 Tesztfájlok 55
6.3.2 Automatikus tesztgenerálás 56
6.4 Teszt végrehajtás 56
6.4.1 Manuális tesztelés 56
6.5.2 Teljesítmény metrikák 57
6.5.3 Hibák és javítások 58

1. Specifikáció

1.1.0 Program célja

A program célja ballisztikai számítások elvégzése különböző lőállások, célpontok és lövegek adatainak felhasználásával.
A rendszer:

- Megállapítja, melyik lőállás van legközelebb egy adott célponthoz.
- Kiszámolja a szükséges lövési szöget és irányt a löveg paramétereinek figyelembevételével.
- Automatikus ajánlást tesz, ha a megadott löveg vagy lőállás nem megfelelő.

1.2.0 Főbb funkciók

- Adatbetöltés szöveges fájlokból (I, L, CLA, CLO, CA formátumban).
- Automatikus célpont–bázis párosítás (legközelebbi lőállás kiválasztása).
- Ballisztikai számítás (távolság, szög, irány, energia).
- Intelligens ajánlás:
  - Ha célpont elérhetetlen → új lőállást vagy löveget ajánl.
  - Ha kettő ugyanolyan távolságú bázis is van →a raktár mennyiség dönt.
- Eredmény log készítése és mentése.
- Interaktív CLI parancsokkal (hozzáadás, törlés, lista, számítás).

1.3.0 Bemeneti adatok formátuma és típusai

A bemenet szöveges fájlokból származik, egy sor egy rekordot tartalmaz.

1.3.1 Általános formátum

I<ID> L<Label> CLA<Latitude> CLO<Longitude> CA<Altitude>;

- **ID**: előjel nélküli egész szám (azonosító)
- **Label**: szöveges címke
- **Latitude, Longitude, Altitude**: lebegőpontos számok (koordináták)

1.3.2 Példa bemenet

I0 LSchonherz CLA47.4729283 CLO19.0531429 CA0.0;

I1 LValami CLA57.476002 CLO29.070322 CA2.2;

1.3.3 Fájltípusok

Bázisok (lőállások): ugyanaz a formátum, plusz lövegkészlettel. A tárolt formátum

--------The defaults----------

[I] : id of base

[L] : label (name)

[CLA] : latitude (degrees)

[CLO] : longitude (degrees)

[CA] : altitude (meters)

--------The parameters----------

[AMS] : ammo-stock { ammo_id:count, ... }

Példa:

I3 LBerlin CLA52.520000 CLO13.405000 CA34.0 AMS : {0:0, 0:0, 7:2500, 18:400, 33:4140, 34:7000, 0:0 };

- **Célpontok**: ugyanaz a formátum, plusz AREA<terület> .

--------The defaults----------

[I] : id of base

[L] : label (name)

[CLA] : latitude (degrees)

[CLO] : longitude (degrees)

[CA] : altitude (meters)

--------The parameters----------

[A] : area (km^2)

Példa:

I14 LDublin CLA53.349800 CLO-6.260300 CA20.0 A117.000;

- **Löveg katalógus**:

--------The defaults----------

[I] : the id of the ammo

[N] : the name string

--------The parameters----------

[W] : the weight

[D] : the diameter (caliber)

[BR] : the blast radius

[MV] : muzzle velocity

[BC] : bullet coefficient

speciális formátum:

I<ID> N<Name> W<Weigth> D<Diameter/Caliber> BR<Blast_radius> MV<Muzzle_velocity> BC<bullet coefficient>;

Példa:

I9 N122_9M521_HE W66.500 D122.000 BR30.000 MV730.000 BC26.000;

1.3.4 Parsing szabályok

- // utáni sorok **kommentek** → figyelmen kívül hagyva.
- /* ... */ blokkok **blokkkommentek** → figyelmen kívül hagyva.
- Az adatok sorrendje **tetszőleges** (kulcsszavak alapján dolgozik).
- A tokenek kis és nagy betűvel való írása nem számít, **normalizálva** lesznek.
- Hibás formátum esetén a sort kihagyja.

1.3.2. Kapcsolatok felépítése

A program minden célponthoz kiszámolja, melyik lőállás van a legközelebb.
Ez alapján létrejön a célpont–lőállás párosítás.

1.3.3. Számítás

A kiválasztott vagy automatikusan választott löveg alapján történik a tényleges számítás:

- távolság a célpont és lőállás között,
- szükséges lövési szög,
- irány,
- energia és hatótáv kiértékelés.

1.3.4 Parsing szabályok

- // utáni sorok **kommentek** → figyelmen kívül hagyva.
- /* ... */ blokkok **blokk kommentek** → figyelmen kívül hagyva.
- Az adatok sorrendje **tetszőleges** (kulcsszavak alapján dolgozik).
- Duplikált azonosítók esetén **normalizálás** történik (egyedi ID-k).
- Hibás formátum esetén a sor **elhagyva**, hibaüzenet kiírva.

1.4.0 Kimeneti adatok

A számítás eredményeit a program **alapból nem menti**, csak megjeleníti.
A felhasználó választhat, hogy szeretné-e menteni az eredményeket.

1.4.1 Alapértelmezett kimenet példa(terminal)

```
Suggested base and ammo:
Base ID: 15  Label: Brussels  Coordinates: lat 50.850300, lon 4.351700, alt 13.00 m
Ammo ID: 45  Label: 400_TB
```

1.4.2 Mentési lehetőségek

Az eredményeket egy log.txt file-ba menti.

- **Új fájlba mentés**: ha nem létezik, akkor létrehozz egyet majd menti.
- **Hozzáfűzés**: ha létezik, akkor az eredmény hozzáfűzése egy létező fájl végéhez.

1.5.0 Követelmények (funkcionális követelmények)

1.5.1 Automatikus ajánlás

- Ha a célpont **nem érhető el** a megadott löveggel/lőállással → új lőállást vagy löveget ajánl.

1.5.2 Adatkezelés

- **Dinamikus adatszerkezetek** (láncolt listák) az adatok tárolására.
- **Memóriaszivárgás elkerülése** (minden lefoglalt memória felszabadítva).
- **Konfigurációs fájl támogatás** (config.txt) a beállítások mentésére.

1.5.3 Hibakezelés

- **Érvénytelen bemenet** esetén hibaüzenet, majd kihagyás.
- **Hiányzó konfig** esetén alapértelmezett konfiguráció létrehozása.
- **Duplikált azonosítók** eseten a program az először beírt adatot priorizálja, keresésnél csak a felhasználónak számít az ID.

1.5.4 Tesztelhetőség

- **Tesztfájlok** különböző esetekre (helyes, hibás, kevert sorrendű bemenet).
- **Automatikus tesztesetek** az interpreter, parser és az adatbázis tesztelésére.

1.6.0 Tesztelés

A program működésének ellenőrzésére külön tesztfájlok készültek.
Példa tesztadat:

```
I0 LTeSt_DaTa0 CLA40.05 CLO20.0 CA-5.023044;
CLO20.0 CA-5.023044 I1 CLA40.05 LTeSt_DaTa1;
```

Ezek a fájlok ellenőrzik:

- hibás vagy kevert sorrendű bemenetek kezelését,
- kommentek figyelmen kívül hagyását,
- duplikált azonosítók javítását,
- számítások helyességét és ajánlási logikát.

1.6.1 Interpreter teszt

- Teszt dokumentum:

```
//------------Input test--------------
//this is a comment
/*
this is a comment too
I0 LTeSt_DaTa CLA40.05 CLO20.0 CA0-5.023044;
*/
I0 LTeSt_DaTa0 CLA40.05 CLO20.0 CA0-5.023044;
CLO20.0 CA0-5.023044        I1 CLA40.05     LTeSt_DaTa1;
```

- Valós eredmény:

```
Starting test...
[Line comment] :
------------Input test--------------
[Line comment] :
this is a comment
[Block comment] :
this is a comment too

I0 LTeSt_DaTa CLA40.05 CLO20.0 CA0-5.023044;
capacity: 10, items: 2
0 TeSt_DaTa0
1 TeSt_DaTa1
---------------------------------
Test succeeded!
---------------------------------
```

2. Adatszerkezetek

2.1.0 Changelog

A programot kibővítettem egy automatikusan generált config.txt file-al, valamint opcionálisan színes karakter megjelenítéssel (windows rendszeren szükséges hozzá a

„reg add HKCU\\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f”

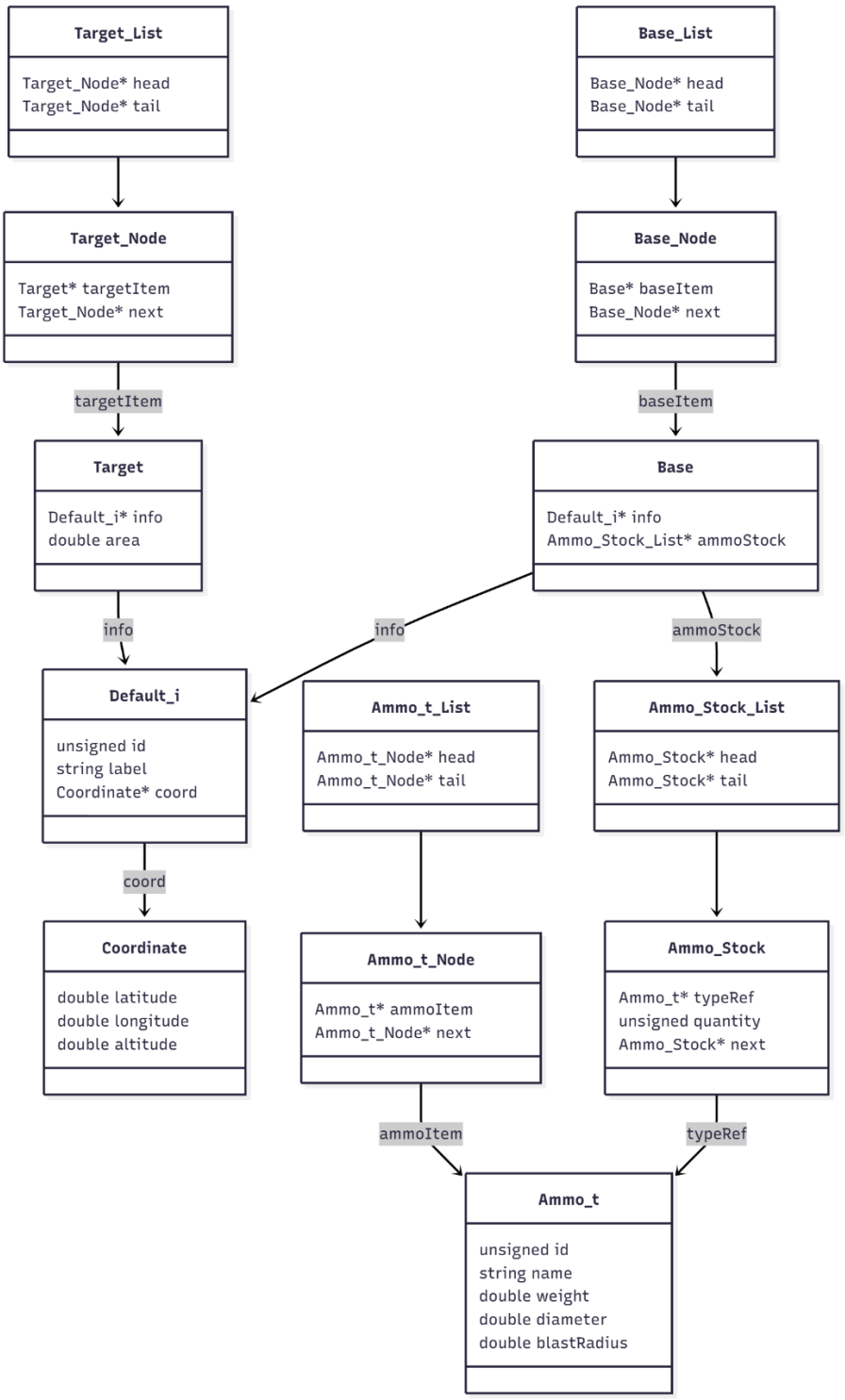
futtatással, mivel a Windows terminál alapból nem támogatja az ANSI escape kódokat, csak virtuális módban), a jobb felhasználói élmény érdekében a CLI parancskészlet bővült, valamint modal loop támogatást kapott. Újra dolgoztam a file betöltő praseret, interpretert, valamint tokenizert. A config file-nak köszönhetően az adat listáknak most már nem kell a munka könyvtárban lenniük.

2.2.0 Adatkezelés

2.2.1 Láncolt strázsált lista

Az adatok tárolása 4 elől-hátul strázsált, hátrafelé láncolt listában történik. Ezek közül 2 fésűs elrendezésben van.

[HEAD] => [elem1] => [elem2] => ... => [TAIL]



Horváth Ádám József
M74IPN

2.3.0 Löveg lista

A löveglista (Ammo_t_List) a program globális lőszer-adatbázisa.
Ez tartalmazza a rendszerben elérhető összes lőszer típus definícióját, amelyeket később a bázisok készletei (Ammo_Stock) referencia alapján hivatkoznak.

A lista ID szerint rendezett, elöl-hátul strázsás láncolt lista.

A globális lőszeradatok a következő információkat tartalmazzák:

- egyedi azonosító
- név
- fizikai paraméterek (tömeg, átmérő, robbanási sugar)

2.3.1 Löveg adatszerkezet

- **Ammo_t** - egyetlen lőszer definíció. Egy lőszer fizikai és azonosítási adatait tartalmazza.
- **Ammo_t_Node** - A lista egy eleme, és egy pointerrel tovább kapcsolódik a listába.
- **Ammo_t_List** - globális lőszerlista, ID-k alapján sorrendben tárolja a típusokat

```
/*Global Ammo type*/
typedef struct Ammo_t {
    unsigned id;
    char* name;         // dynamically allocated string
    double weight;
    double diameter;
    double blastRadius;
} Ammo_t;

/*Ammo_t_Node for linked list of global ammo*/
typedef struct Ammo_t_Node {
    Ammo_t* ammoItem;
    struct Ammo_t_Node* next;
} Ammo_t_Node;

/*Global Ammo list*/
typedef struct {
    Ammo_t_Node* head;
    Ammo_t_Node* tail;
} Ammo_t_List;
```

2.3.2 Löveg adatszerkezet főbb függvényei

```
// --- Ammo List, ordered by id ---
void initAmmo_t_List(Ammo_t_List* list);
Ammo_t_Node* addAmmo_t(Ammo_t_List* list, Ammo_t* ammo);
int deleteAmmo_t(Ammo_t_List* list, unsigned id);
Ammo_t_Node* popFrontAmmo_t(Ammo_t_List* list);
Ammo_t_Node* popBackAmmo_t(Ammo_t_List* list);
void freeAmmo_t_List(Ammo_t_List* list);
```

initAmmo_t_List

- üres listát készít
- head = tail = NULL
- a globális ammo-lista létrehozásakor hívódik meg

addAmmo_t

- új lőszert ad a listához
- ID szerint rendezett beszúrás
- visszatér az új csomópont pointerével

deleteAmmo_t

- töröl egy lőszert azonosító alapján
- ha nem létezik → 0
- ha sikerült → 1

popFrontAmmo_t / popBackAmmo_t

- sor-szerű vagy verem-szerű felhasználáshoz
- elsősorban CLI parancsokhoz vagy teszteléshez

freeAmmo_t_List

- felszabadítja az összes csomópontot és a dinamikus neveket

2.4.0 Bázisok, Célpontok

A bázisok és a célpontok közös alapja a Default_i struktúra, amely egységesíti az azonosítást és a koordinátakezelést.
A differenciák a bázis-specifikus és célpont-specifikus adatmezőkben jelennek meg.

2.4.1 Alap pozíció adatok

Default_i - egységes pozíció adatszerkezet, általános paraméter adatokat tartalmaz

```
/*Default_i (shared by Base and Target)*/
typedef struct {
    unsigned id;
    char* label;        // dynamically allocated string
    Coordinate* coord;  // pointer to Coordinate
} Default_i;
```

Szerepe:

Egy egységes struktúra meghatározása:

- egyedi azonosító
- név/jelző
- pozíció (Coordinate)
- duplikáció minimalizálása

2.4.2 Bázis lista

Ammo_Stock - bázisokra jellemző lőszerkészlet, Ammo_t * tartalmaz

```
/*Base-specific Ammo Stock*/
typedef struct Ammo_Stock {
    Ammo_t* typeRef;           // pointer to global ammo
    unsigned quantity;
    struct Ammo_Stock* next;
} Ammo_Stock;
```

Szerepe:

- A bázison tárolt lőszerek mennyiségét reprezentálja.

Ammo_Stock_List - strázsás lista lőszerkészletre. Minden bázis saját, külön lőszerkészletet tart fent.

```
/*Ammo stock list for each Base*/
typedef struct {
    Ammo_Stock* head;
    Ammo_Stock* tail;
} Ammo_Stock_List;
```

Base - egyetlen bázis adatai

```
/*Base struct*/
typedef struct Base {
    Default_i* info;
    Ammo_Stock_List* ammoStock;
} Base;
```

A bázis:

- rendelkezik pozícióval és névvel (info)
- saját lőszerraktárral (ammoStock)

Base_Node - A lista egy eleme

```
/*Base_Node for linked list*/
typedef struct Base_Node {
    Base* baseItem;
    struct Base_Node* next;
} Base_Node;
```

Base_List — bázisok tárolása

```
typedef struct {
    Base_Node* head;
    Base_Node* tail;
} Base_List;
```

Szerepe:

- A program futása alatt a bázis tárolására
- Futási idő végén a statikus lista módosítására szolgál

2.4.3 Célpontok

A célpontok szerkezete egyszerűbb, mert nem rendelkeznek készlettel és dinamikus tulajdonságokkal.

Target - egy célpont adatai

```
/*Target struct*/
typedef struct Target {
    Default_i* info;
    double area;
} Target;
```

A célpont:

- rendelkezik pozícióval (info->coord)
- rendelkezik megsemmisítendő területtel (area)
- a ballisztikai számítások ezt használják a szükséges lőirány meghatározásához

Target_Node - A lista egy eleme

```
/*Target_Node for linked list*/
typedef struct Target_Node {
    Target* targetItem;
    struct Target_Node* next;
} Target_Node;
```

Target_List - A célpont lista

```
typedef struct {
    Target_Node* head;
    Target_Node* tail;
} Target_List;
```

A célpontlista:

- betöltéskor feltöltődik
- CLI alatt módosítható (insert/delete)
- select_target innen választ
- bulk mód itt iterál végig ID szerint

2.4.4 Célpontok adatszerkezetek főbb függvényei

```
// --- Base List, ordered by id ---
void initBase_List(Base_List* list);
Base_Node* addBase(Base_List* list, Base* base);
int deleteBase(Base_List* list, unsigned id);
Base_Node* popFrontBase(Base_List* list);
Base_Node* popBackBase(Base_List* list);
void freeBase_List(Base_List* list);

// --- Ammo Stock List (per Base), ordered by ammo id ---
void initAmmo_Stock_List(Ammo_Stock_List* list);
Ammo_Stock* addAmmo_Stock(Ammo_Stock_List* list, Ammo_t* ammoRef,
unsigned quantity);
int deleteAmmo_Stock(Ammo_Stock_List* list, Ammo_t* ammoRef);
Ammo_Stock* popFrontAmmo_Stock(Ammo_Stock_List* list);
Ammo_Stock* popBackAmmo_Stock(Ammo_Stock_List* list);
void freeAmmo_Stock_List(Ammo_Stock_List* list);

// --- Target List, ordered by id ---
void initTarget_List(Target_List* list);
Target_Node* addTarget(Target_List* list, Target* target
int deleteTarget(Target_List* list, unsigned id);
Target_Node* popFrontTarget(Target_List* list);
Target_Node* popBackTarget(Target_List* list);
void freeTarget_List(Target_List* list);
```

- **initBase_List(Base_List* list)**

  A lista inicializálása.

  - Üres állapotba teszi (head, tail NULL, size = 0).
  - Minden további művelet csak ezután használható biztonságosan.

- **addBase(Base_List* list, Base* base)**

  Új bázis beszúrása rendezett helyre, tehát:

  - megkeresi azt a pozíciót, ahol az ID alapján lennie kell
  - létrehoz egy új csomópontot
  - beilleszti úgy, hogy a lista továbbra is rendezett marad

  Visszatérési érték: a létrehozott csomópont pointere.

- **deleteBase(Base_List* list, unsigned id)**

  Egy bázis törlése id alapján:

  - végigmegy a listán
  - ha talál pontos ID-t, kiveszi a csomópontot
  - felszabadítja
  - visszatér 0/1 vagy error kóddal

- **popFrontBase(Base_List* list)**

  Az első elemet kiveszi a listából:

  - head => head->next
  - leválasztja a régi head-et
  - visszaadja a kicsipkedett csomópontot (caller feladata felszabadítani)

  Tipikusan akkor kell, ha fájlból töltve végig akarod pucolni a listát.

- **popBackBase(Base_List* list)**

  Az utolsó elemet veszi ki:

  - végigmegy a lista végére
  - leválasztja a tail-t
  - visszaadja a csomópontot

  Ez lassabb egy sima egyszerű listában, de néha szükséges.

- **freeBase_List(Base_List* list)**

  Az egész lista felszabadítása:

  - végigmegy a csomópontokon
  - free() minden Base-re és node-ra
  - végül üres állapotba állítja

  Ez a helyes lezárás a program végén.

- **initAmmo_Stock_List**

  Üresre állítja a készletlistát.

- **addAmmo_Stock**

  Hozzáad egy új ammo tételt a megfelelő helyre, rendezve.
  Mivel ammoRef az ammo definíció pointere, a lista nem tárol másolatot, csak hivatkozást + quantity értéket.

- **deleteAmmo_Stock**

  Eltávolít egy adott ammo fajtát id alapján.
  Ez valójában az ammoRef->id szerint működik.

- **popFrontAmmo_Stock / popBackAmmo_Stock**

  Ugyanaz, mint a Base esetében - csomópontot kicsip, visszadobja a hívónak.

- **freeAmmo_Stock_List**

  Törli az adott Base saját ammo-készletét.

- **initTarget_List**

  Üres lista létrehozása.

- **addTarget**

  Új célpont hozzáadása id szerinti sorrendben.
  Ez fontos a gyorsabb keresés, átláthatóság és stabil output miatt.

- **deleteTarget**

  Célpont törlése id alapján.

- **popFrontTarget / popBackTarget**

  Az első / utolsó célpont kicsipése.

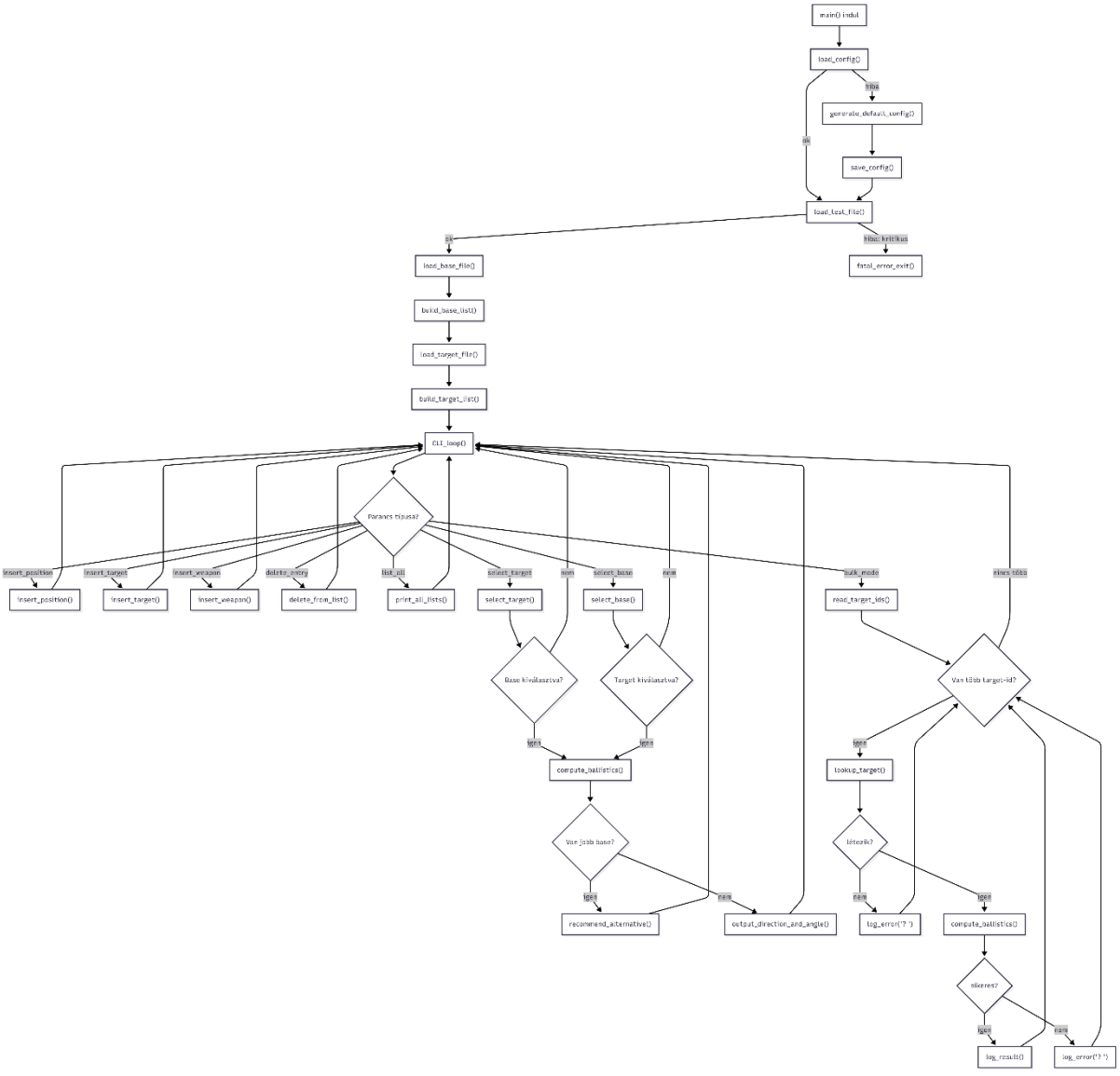
- **freeTarget_List**

  Az összes célpont törlése.

2.5.0 Program flowchart

A program négy fő lépésből áll:

- Inicializáció és rendszerelőkészítés
- Konfiguráció betöltése vagy létrehozása
- Parancsértelmezés és adatkezelés
- Tiszta kilépés és memóriafelszabadítás



2.5.1 Rendszerindítás és inicializáció

A program indulásakor az első feladat a rendszer előkészítése. Ez azt jelenti, hogy létrehozzuk a szükséges adatszerkezeteket, lefoglaljuk a memóriát, és alaphelyzetbe állítjuk az állapotváltozókat.
Ez biztosítja, hogy a bázislista, a lőszerkészletek és a célpontlista mindig következetes állapotban legyenek.

Ez az első lépés minden további funkcióhoz szükséges, és garantálja, hogy a program mindig ugyanúgy induljon el.

2.5.2 Konfiguráció betöltése

Ezután a program megpróbálja betölteni a korábbi beállításokat a konfigurációs fájlból.

- Ha sikerül: a program a fájlban található adatokkal folytatja a működést.
- Ha nem sikerül: a program létrehoz egy új, alapértelmezett konfigurációt. Ilyenkor a felhasználó dönthet, hogy menti-e az új beállításokat, vagy ideiglenesen futtatja a programot a létrehozott értékekkel.

Így mindig lesz használható konfiguráció, akár új, akár meglévő adatok alapján.

2.5.3 Parancsértelmezés és fő vezérlési ciklus

A program központi része a folyamatos parancsértelmező ciklus, ami a felhasználói vagy rendszer által indított parancsokat kezeli.

A folyamat:

- **Parancs beolvasása** - a felhasználó vagy a program indítja.
- **Érvényesség ellenőrzése** – a parancs helyes-e formailag és tartalmilag.
- **Parancsfeldolgozás** - érvényes parancsok meghívják a megfelelő modulokat, például: új bázis hozzáadása, bázis törlése, lőszerkészlet módosítása, célpontok kezelése, listák kiíratása.
- **Hibakezelés** - érvénytelen parancs esetén a program jelzi a hibát és új bemenetet kér.

A ciklus addig fut, amíg a felhasználó ki nem adja a kilépés parancsot.

2.5.4 Adatszerkezetek kezelése

- A program 4 listát kezel: bázisok és lőszerkészletek, célpontok, lőszer specifikációk.
- Az adatszerkezetek külön működnek, de a vezérlés egységesen történik.

2.5.5. Kilépés és memóriafelszabadítás

A kilépéskor a program tisztán zárja le a működését:

- Minden lista felszabadítása
- Opcionális konfiguráció mentése
- Erőforrások lezárása

Ez megakadályozza, hogy a program memóriaszivárgást okozzon.

---

3. Főbb algoritmusok

3.1.0 Adatbeolvasás és feldolgozás algoritmusa

3.1.1 Tokenizálás és elemzés

Input: szöveges sor (pl.: "I0 LSchonherz CLA47.4729283 CLO19.0531429 CA0.0;")

Output: strukturált adat (Base vagy Target)

Algoritmus lépései:

1. Sor beolvasása fájlból
2. Kommentek szűrése:
   - Ha sor "//"-el kezdődik → kihagyás
   - Ha "/*" és "*/" között van → kihagyás
3. Tokenek szétválasztása " " (alapból szóköz) elválasztóval
4. Minden token elemzése:
   - Ha "I"-vel kezdődik → ID érték kinyerése
   - Ha "L"-lel kezdődik → Label érték
   - Ha "CLA"-val → latitude
   - Ha "CLO"-val → longitude
   - Ha "CA"-val → altitude
   - Ha "AMS"-sel → lőszerkészlet
   - Ha "A"-val → terület (célpontnál)
5. Struktúra létrehozása a kinyert adatokkal

3.1.2 Scan függvény

ALGORITMUS: scan(filename, handler, data, mode)

BEMENET: fájlnév, kezelőfüggvény, adat, mód (DEBUG/RUNNING)

KIMENET: HRESULT (siker/hiba)

LÉPÉSEK:

1. Fájl megnyitása olvasásra
2. Karakterenkénti feldolgozás (state machine):

ÁLLAPOTOK:

- NORMAL: alapállapot
- LINE_COMMENT: "//" utáni komment
- BLOCK_COMMENT: "/* ... */" blokkomment
- RECORD: adatrekord feldolgozása

3. Állapotátmenetek:

- '/' karakter → komment kezdetének ellenőrzése
- Nem whitespace karakter → RECORD állapot
- ';' karakter → rekord vége, handler meghívása

4. Handler feldolgozza a rekordot és visszaadja az eredményt
5. Fájl bezárása

3.1.3 Tokenizálás (strtoken használatával)

ALGORITMUS: handle_base() / handle_target() / handle_ammo_t()

BEMENET: karakterlánc (rekord)

KIMENET: strukturált adat (Base/Target/Ammo_t)

LÉPÉSEK:

1. strtoken() segítségével tokenekre bontás
2. Minden token típusának azonosítása:
   - "I" → ID
   - "L" → Label
   - "CLA" → Latitude
   - "CLO" → Longitude
   - "CA" → Altitude
   - "A" → Area (csak Target)
   - "AMS" → Ammo Stock (csak Base)
   - "N", "W", "D", "BR", "MV", "BC" → Ammo paraméterek

3. Értékek kinyerése tokenekből
4. Struktúra létrehozása dinamikus memóriafoglalással
5. Hozzáadás a megfelelő listához

3.2.0 Ballisztikai számítások

3.2.1 Koordináta transzformációk

ALGORITMUS: wgs84_to_ecef(lat_deg, lon_deg, h)

BEMENET: WGS84 koordináták (fok, méter)

KIMENET: ECEF (Earth-Centered, Earth-Fixed) vektor

MATEMATIKAI LÉPÉSEK:

1. Átalakítás radiánba: $\phi = lat \times \pi/180$, $\lambda = lon \times \pi/180$
2. Segédértékek: $\sin\phi$, $\cos\phi$, $\sin\lambda$, $\cos\lambda$
3. Normál sugár: $N = a / \sqrt{(1 - e^2 \times \sin^2\phi)}$
   ahol: $a = 6378137.0$ (WGS84_A), $e^2 = 6.69437999014e-3$
4. ECEF koordináták:
   $x = (N + h) \times \cos\phi \times \cos\lambda$
   $y = (N + h) \times \cos\phi \times \sin\lambda$
   $z = (N \times (1 - e^2) + h) \times \sin\phi$

3.2.2 ENU (East-North-Up) transzformáció

ALGORITMUS: ecef_to_enu(p, origin, lat0_deg, lon0_deg)

BEMENET: ECEF pont, referencia origo, referencia szélesség/hosszúság

KIMENET: ENU koordináták (kelet, észak, magasság)

LÉPÉSEK:

1. Különbségvektor: dx = p.x - origin.x, stb.
2. Forgatómátrix elemei:
   - $\sin\phi_0$, $\cos\phi_0$, $\sin\lambda_0$, $\cos\lambda_0$
3. ENU transzformáció:
   $East = -\sin\lambda_0 \times dx + \cos\lambda_0 \times dy$
   $North = -\sin\phi_0 \times \cos\lambda_0 \times dx - \sin\phi_0 \times \sin\lambda_0 \times dy + \cos\phi_0 \times dz$
   $Up = \cos\phi_0 \times \cos\lambda_0 \times dx + \cos\phi_0 \times \sin\lambda_0 \times dy + \sin\phi_0 \times dz$

3.2.3 RK4 (Runge-Kutta 4. rendű) numerikus integráció

ALGORITMUS: rk4(state, dt, BC)

BEMENET: aktuális állapot, időlépés, ballisztikai együttható

KIMENET: új állapot (pozíció, sebesség)

LÉPÉSEK:

1. k1 = derivatíva(current state)
2. k2 = derivatíva(state + dt/2 × k1)
3. k3 = derivatíva(state + dt/2 × k2)
4. k4 = derivatíva(state + dt × k3)
5. Új állapot = régi + dt/6 × (k1 + 2k2 + 2k3 + k4)

DERIVATÍVA SZÁMÍTÁS:

- Sebesség: $v = \sqrt{(vx^2 + vy^2 + vz^2)}$
- Légellenállás: Cd = g7_drag_coefficient(v)
- Drag = $0.5 \times \rho \times Cd \times v^2 / BC$ ($\rho = 1.225$ kg/m^3)
- Gyorsulás: ax = -drag × vx/v, ay = -drag × vy/v, az = -g - drag × vz/v

3.2.4 Ballisztikai megoldás keresése

ALGORITMUS: solve_ballistic(targetENU, muzzleVel, BC)

BEMENET: cél ENU koordináták, kezdősebesség, BC

KIMENET: BallisticResult (emelkedési szög, irány, repülési idő)

LÉPÉSEK:

1. Irány (azimut) számítás: atan2(kelet, észak)
2. Vízszintes távolság: $R = \sqrt{(kelet^2 + észak^2)}$
3. Magasságkülönbség: DZ = fel
4. Szögkeresés 5°-tól 80°-ig 0.1°-os lépésekben:
   a. Kezdeti sebességkomponensek számítása
   b. RK4 integráció időlépésekben
   c. Talajérintés detektálás (z < DZ)
   d. Hibaszámítás: |aktuális távolság - R|
   e. Legjobb megoldás tárolása

3.3.0 Regionális keresési algoritmus (calculations.c)

3.3.1 Region definition

REGION LOGIKA:

- Region = ID / 1000 (egész osztás)

- Példák:

- ID 0-999 → Region 0
- ID 1000-1999 → Region 1
- ID 2000-2999 → Region 2

3.3.2 Region-aware keresés (kétfázisú)

ALGORITMUS: SuggestBestBaseAndAmmo_RegionAware(list, target, outBase, outAmmo)

BEMENET: bázis lista, célpont, kimeneti pointerek

KIMENET: 1 (siker) vagy 0 (hiba)

LÉPÉSEK:

1. Célpont régió meghatározása: target_region = target->id / 1000

2. PASS 1: Saját regió keresése

Minden bázisra:

a. b_region = b->id / 1000

b. Ha b_region != target_region → SKIP

c. Távolság számítás (GeoDistance3D)

d. Ha távolság < 100 m → SKIP (ugyanazon pozíció)

e. Löveg keresése a bázis készletében

f. Ballisztikai számítás (compute_fire_solution)

g. Ha valid → legjobb tárolása

3. PASS 2: Más regionok keresése (csak ha PASS 1 nem talált)

- Minden regionra, amely nem target_region
- Region header kiírása első alkalommal
- Ugyanaz a keresési logika, mint PASS 1-ben

4. Kritériumok:

- Elsődleges: minimális 3D távolság
- Döntetlen esetén: nagyobb lőszer mennyiség
- Tűrés: 1.0 m (float pontosság)

3.3.3 Távolság számítások

ALGORITMUS: GeoDistance3D(a, b)

BEMENET: két Coordinate struktúra

KIMENET: 3D távolság méterben

LÉPÉSEK:

1. 2D távolság (Haversine):
   a. ϕ_1, ϕ_2 = latitude × π/180
   b. $\Delta\phi = \phi_2 - \phi_1$
   c. $\Delta\lambda = (\text{longitude}_2 - \text{longitude}_1) \times \pi/180$
   d. $a = \sin^2(\Delta\phi/2) + \cos(\phi_1) \times \cos(\phi_2) \times \sin^2(\Delta\lambda/2)$
   e. $c = 2 \times \text{atan2}(\sqrt{a}, \sqrt{(1-a)})$
   f. distance_2d = R × c (R = 6378137 m)

2. Magasságkülönbség:
   $\Delta h = \text{altitude}_2 - \text{altitude}_1$

3. 3D távolság:
   $\sqrt{(\text{distance_2d}^2 + \Delta h^2)}$

3.4.0 Láncolt lista kezelési algoritmusok

3.4.1 Strázsált lista struktúra

LISTA STRUKTÚRA:

- head: dummy node (üres)
- tail: dummy node (üres)
- Valós elemek: head->next ... tail->prev között
- Előny: egységes beszúrás/törlés, nincs speciális eset kezelés

3.4.2 Rendezett beszúrás (ID alapján)

ALGORITMUS: add_Base(list, base) / add_Target() / add_Ammo_t()
BEMENET: lista, új elem
KIMENET: hozzáadott node pointer

LÉPÉSEK:
1. Járó pointer inicializálása: current = list->head
2. Keress helyet: amíg current->next != list->tail ÉS
   current->next->item->id < new_item->id
   current = current->next
3. Új node létrehozása dinamikus memóriafoglalással
4. Beszúrás:
   new_node->next = current->next
   current->next = new_node
5. Memóriamásolás: érték másolás koordinátákhoz

3.4.3 Keresés optimalizált úton

ALGORITMUS: FindBaseByID(list, id) / FindTargetByID() / FindAmmoByID()
BEMENET: lista, keresett ID
KIMENET: elem pointer vagy NULL

OPTIMALIZÁCIÓ:
1. Lista üres? → NULL
2. Irány meghatározása:
   dir = (|első_elem_id - keresett_id| ≤ |utolsó_elem_id - keresett_id|) ? 0 : 1
   - 0: elejéről kezdve keres
   - 1: végétől kezdve keres
3. Lineáris keresés a kiválasztott irányban

3.4.4 Memóriafelszabadítás (mélységi bejárással)

ALGORITMUS: free_Base_List(list) / free_Target_List() / free_Ammo_t_List()

BEMENET: lista pointer pointer

KIMENET: HRESULT

LÉPÉSEK:

1. Amíg lista nem üres:
   a. Utolsó elem kiszedése (popBack)
   b. Elem felszabadítása:
   - free(item->label) // dinamikus sztring
   - free(item) // maga a struktúra
   - free(node) // lista node
2. Head és tail dummy node-ok felszabadítása
3. Lista struktúra felszabadítása
4. Pointer NULL-ra állítása

3.5.0 Konfiguráció kezelés algoritmusa (load.c)

3.5.1 Konfig fájl generálás

ALGORITMUS: generate_config(configfile, config_file)

BEMENET: fájl pointer, fájlnév

KIMENET: HRESULT

LÉPÉSEK:

1. Felhasználói input:
   - Munkakönyvtár bekérése
   - Színmód választás (színes/monokróm)
2. Fájl létrehozás írásra
3. Konfigurációs adatok kiírása:
   - DIR: "mappa/elérési/út/"
   - COLOUR_M: 0 vagy 1
4. Dokumentációs kommentek hozzáadása
5. Fájl bezárása

3.5.2 Konfig fájl betöltés

ALGORITMUS: read_config(config_file)

BEMENET: fájlnév

KIMENET: HRESULT

LÉPÉSEK:

1. Fájl megnyitása olvasásra
2. scan() hívása handle_config handlerrel
3. Handler feldolgozása:
    - "DIR:" token → mappaelérési út beállítása
    - "COLOUR_M:" token → színmód beállítása
4. Fájl bezárása
5. Beállított értékek alkalmazása a programra

3.6.0 Hibakezelés és naplózás algoritmusai

3.6.1 Hibajelentés formázás

ALGORITMUS: format_basic_error_string(error_buffer, error_flags)

BEMENET: hibapuffer, hibajelző tömb

KIMENET: HRESULT

LÉPÉSEK:

1. error_flags tömb bejárása (5 elem)
2. Minden pozícióra:
   - Ha flag != 1 → hiányzó paraméter
   - Megfelelő token hozzáfűzése:
    - F_ID (0) → "I"
    - F_LABEL (1) → "L"
    - F_LATITUDE (2) → "CLA"
    - F_LONGITUDE (3) → "CLO"
    - F_ALTITUDE (4) → "CA"
3. Vesszővel elválasztás többszörös hiány esetén
4. String lezárása nullával

3.6.2 Naplózás fájlba

ALGORITMUS: LogResult(base, target, ammo, result)

BEMENET: bázis, célpont, löveg, számítási eredmény

KIMENET: void (fájlba ír)

LÉPÉSEK:

1. Időbélyeg generálás: time(), ctime()
2. Fájl megnyitása hozzáfűzésre (append)
3. Formázott kiírás:

- Időbélyeg
- Bázis adatok (ID, név, koordináták)
- Célpont adatok (ID, név, koordináták, terület)
- Löveg paraméterek (összes fizikai tulajdonság)
- Számítási eredmény (szög, irány, repülési idő, sebesség)

4. Fájl bezárása

3.6.3 Eredménymentés formázás

ALGORITMUS: Save_BaseList_file(fp) / Save_TargetList_file() / Save_Ammo_t_List_file()

BEMENET: fájl pointer

KIMENET: void (fájlba ír)

LÉPÉSEK:

1. Dokumentációs fejléc kiírása (komment formátumban)
2. Lista bejárása:
   a. Region header kiírása region változásakor
   b. Minden elem formázott kiírása:
      - Base: ID, név, koordináták, lőszerkészlet lista
      - Target: ID, név, koordináták, terület.
      - Ammo: összes fizikai paraméter.
   c. Formátum: az eredeti bemeneti formátumnak megfelelően.
3. Region header funkció: GetRange(id) alapján csoportosítás.

4. Felhasználói dokumentáció

4.1.0 Telepítés és indítás

4.1.1 Fordítás

Windows:

gcc calculations.c color.c commands.c debug.c error.c helpers.c load.c main.c storage.c -lm -Wall -Wextra -o ballistic_trajectory_calculator.exe

Linux/macOS:

gcc calculations.c color.c commands.c debug.c error.c helpers.c load.c main.c storage.c -lm -Wall -Wextra -o ballistic_trajectory_calculator.exe

4.1.2 ANCI színek engedélyezése (Windows)

A színes kimenet Windows alatt csak virtuális terminál módban működik:

cmd
reg add HKCU\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f

4.2 Kezdeti beállítás

4.1.1 Első indításkor

1. A program megkérdezi: "Failed to open or locate [config.txt]"
2. Válaszd: yes (új konfiguráció létrehozása)
3. Add meg a munkakönyvtár elérési útját
(pl.: C:/Users/felhasznalo/ballistic_data/)
4. Válaszd ki a színmódot: colour (színes) vagy mono (fekete-fehér)

4.3.0 Főmenü és alapvető működés

4.3.1 Indítás utáni képernyő

```
==================================
Ballistic trajectory calculator
Created by.: Adam Jozsef Horvath
==================================

Type [exit] to leave or type [help] for the commands!

----------------command list----------------
  back   - step back
  cancel - Cancel event
  select - select choice
  list   - list values
  add    - add value
  delete - delete value
  save   - save changes
  bases  - List bases
  targets- List targets
  ammo   - List ammo
  calc   - calculate trajectory
  test   - Do a manual self-test
  clear  - Clears the terminal
  config - Modify config
  exit   - Exit program
  help   - Show commands
  colour - coloured mode command
  mono   - monochrome mode command
  yes    - yes command
  no     - no command
------------------------------------------
>
```

4.4.0 Parancsok részletes leírása

4.4.1 Adatkezelési parancsok

bases - Bázisok kezelése

> bases

What would you like to do? <[back]>

- show base list [list]
- add base to list [add]
- remove base from list [delete]

Példa használat:

> bases

> list # Bázisok listázása

> add # Új bázis hozzáadása (formátum: I0 LBerlin CLA52.520000 CLO13.405000 CA34.0 AMS:{1:100,2:50};)

> delete 5 # 5-ös ID-jű bázis törlése

> back # Vissza a főmenübe

targets - Célpontok kezelése (ugyanaz a felépítés, mint a bázisoknál)

> targets

What would you like to do? <[back]>

- show target list [list]
- add target to list [add]
- remove target from list [delete]

ammo - Löveg típusok kezelése

> ammo

What would you like to do? <[back]>

- show ammo type list [list]
- add ammo type to list [add]
- remove ammo type from list [delete]

4.4.2 Számítási parancsok

calc - Számítás indítása

```
> calc
What would you like to do? <[back]>
  - select parameters          [select]
  - work with bases            [bases]
  - work with target           [target]
  - work with ammo             [ammo]
```

Számítási folyamat:

```
> calc
> select
Please input the base, target, ammo ID!
If you aren't sure about the ID's, type [back], and check the [list] for the desired value!
If you can't find, what you are looking for, then [add] it to the list!

> 15 8 42   # base_id=15, target_id=8, ammo_id=42
```

4.4.3 Rendszerparancsok

test - Automatikus teszt futtatása

- Ellenőrzi az összes funkciót és adatbetöltést.

clear - Képernyő törlése

- Csak színes módban működik.

config - Konfiguráció újra generálása

- Új mappaelérési út és színmód beállítása.

exit - Kilépés

```
> exit
Do you want to save before leaving? [YES/NO]
> yes   # Mentés és kilépés
> no    # Kilépés mentés nélkül
```

help - Segítség megjelenítése

- Az aktuális kontextusban elérhető parancsok listája.

4.5.0 Adatfájlok formátuma

4.5.1 Konfigurációs fájl (config.txt)txt

DIR: "C:/Users/felhasznalo/ballistic_data/";

COLOUR_M: 1;

4.5.2 Bázis fájl (bases.txt)

I3 LBerlin CLA52.520000 CLO13.405000 CA34.0 AMS:{0:0, 0:0, 7:2500, 18:400, 33:4140, 34:7000, 0:0};

4.5.3 Célpont fájl (targets.txt)

I14 LDublin CLA53.349800 CLO-6.260300 CA20.0 A117.000;

4.5.4 Löveg katalógus (ammo_catalouge.txt)

I9 N122_9M521_HE W66.500 D122.000 BR30.000 MV730.000 BC26.000;

4.6 Kimeneti formátumok

4.6.1 Képernyőn megjelenő eredmény

Suggested base and ammo:

Base ID: 15 Label: Brussels Coordinates: lat 50.850300, lon 4.351700, alt 13.00 m

Ammo ID: 45 Label: 400_TB

4.6.2 Naplófájl (log.txt)

=== ENTRY @ Thu Jan 15 14:30:22 2025 ===

Base:

ID: 15

Label: Brussels

Coord: latitude 50.850300, longitude 4.351700, altitude 13.00 m

Target:

ID: 8

Label: Target_Alpha

Coord: latitude 48.856613, longitude 2.352222, altitude 35.00 m

Ammo:

ID: 42
Name: 155mm_HE
Weight: 43.000 kg
Diameter: 155.000 mm
BlastRadius: 25.000 m
MuzzleVel: 827.000 m/s
BC(G7): 0.350

Solution:
Elevation: 42.7324 deg
Azimuth: 89.1234 deg
TOF: 128.4321 s
ImpactVel: 312.5432 m/s
Valid: YES

4.7.0 Regionális keresési funkció

A program regionális keresést alkalmaz:
Region 0: ID 0-999
Region 1: ID 1000-1999
Region 2: ID 2000-2999 stb.
A keresés két fázisban történik:
Első fázis: csak a célpont régiójában lévő bázisok
Második fázis: minden más régió (ha az első nem talált megoldást)
Searching region 0-999 (target region)...
Searching other regions (outside 0-999)...
Scanning region 1000-1999...
Scanning region 2000-2999...

4.8.0 Hibakezelés és hibajelentések

4.8.1 Gyakori hibák és megoldások

"Failed to open or locate [fájlnév]"
OK: Ellenőrizd, hogy a fájl a config.txt-ben megadott mappában van-e
OK: Ellenőrizd a fájlnevet (kis/nagybetű érzékeny)
MEGOLDÁS: Használd a config parancsot új mappa beállításához

"Unknown command: [parancs]"

OK: Ellenőrizd a parancs helyesírását

OK: A parancs nem elérhető aktuális kontextusban

MEGOLDÁS: Használd a help parancsot az elérhető parancsok listázásához

"No suitable base/ammo combination found"

OK: A célpont nem érhető el egyetlen bázisból sem

MEGOLDÁS:

Adj hozzá új bázist (bases → add)

Adj hozzá új löveget (ammo → add)

Próbálj másik célpontot

"Memory allocation failed"

OK: Nincs elég memória

MEGOLDÁS: Csökkentsd a betöltött adatok mennyiségét

4.8.2 Hibakeresési mód

Debug információ engedélyezése:

> test

4.9.0 Példa teljes munkamenet

1. Indítás

2. Konfiguráció (első indításkor)

> yes # új konfig létrehozása

C:/Users/én/ballistic/ # mappaelérési út

> colour # színes mód

3. Bázisok ellenőrzése

> bases

> list # meglévő bázisok megtekintése

> back

4. Célpont kiválasztása

> targets

> list # célpontok listája

> back

```
# 5. Számítás indítása
> calc
> select
> 15 8 42            # base_id=15, target_id=8, ammo_id=42
# 6. Kilépés
> exit
> yes               # változtatások mentése
```

4.10.0 Gyorsbillentyűk és tippek

Gyors műveletek:

clear - Gyors képernyőtörlés

help - Gyors segítség bármikor

list - Gyors listázás bármelyik menüben

Hasznos tippek:

Mindig ellenőrizd az ID-ket a list paranccsal

A back paranccsal bármikor visszaléphetsz egy szintet

Minden változtatást menteni kell (save), különben elveszik kilépéskor

---

5. Programozói dokumentáció

5.1 Projekt struktúra és függőségek

5.1.1 Fájlszerkezet

```
ballistic_project/
├── main.c           # Főprogram, inicializáció, fő ciklus
├── commands.c/h     # Parancsértelmezés, CLI kezelés
├── load.c/h         # Fájlbeolvasás, konfiguráció, parser
├── storage.c/h      # Adatszerkezetek, lista műveletek
├── calculations.c/h # Ballisztikai számítások
├── debug.c/h        # Tesztelés, hibakeresés
├── error.h          # Hibakezelés definíciók
├── color.h          # Színes kimenet kezelése
└── debugmalloc.h    # Memóriaszivárgás detektálás
```

5.1.2 Főbb függőségek

Standard C könyvtár: stdio, stdlib, string, math

Saját modulok: kölcsönös függőség minimalizálva

Külső függőségek: nincs (pure C)

5.2.0 Főbb modulok és funkcióik

5.2.1 main.c - Főprogram

Felelősség: Program indítása, fő ciklus, erőforrás kezelés

Főbb függvények:

```
PROCESS Setup();       // Inicializáció, konfig betöltés, adatok betöltése
void CleanUp();        // Memória felszabadítás, mentés
int main(void);        // Főprogram belépési pont
```

Működés:

Setup() hívása - konfiguráció és adatok betöltése

Fő ciklus - felhasználói parancsok feldolgozása

CleanUp() hívása - tisztítás kilépés előtt

5.2.2 commands.c/h - Parancsértelmezés

Felelősség: Felhasználói interfész, parancsok kezelése

Adatszerkezetek:

```
typedef struct {
   char* name;         // Parancs neve ("exit", "help", stb.)
   Command_t type;     // Parancs típusa (enum)
   CommandHandler handler;// Kezelőfüggvény
   char* description;   // Leírás (help számára)
   Command_case case_mask;// Hol használható (kontextus maszk)
} Command;
```

Főbb függvények:

```
Command_t GetCommand(Command_case case_flags, PROCESS *running);
// Felhasználói input olvasása és feldolgozása

Command_t ParseCommandFromBuffer(char *input, Command_case case_flags,
PROCESS *running);
// Bufferből történő parancs feldolgozás
```

void cmd_help(...); // Help parancs kezelése
void cmd_exit(...); // Kilépés kezelése
void cmd_base(...); // Bázisok kezelése
void cmd_target(...); // Célpontok kezelése
void cmd_ammo(...); // Löveg típusok kezelése
void cmd_calculate(...); // Számítás kezelése
Parancskontextusok (case_mask):
CMD_C_GENERIC - Általános parancsok
CMD_C_LOAD - Adatbetöltés mód
CMD_C_CALC - Számítás mód
CMD_C_SYSTEM - Rendszerparancsok
CMD_C_YESNO - Igen/nem választás
CMD_C_BACK - Visszalépés lehetősége
CMD_C_LOOP - Ciklusban maradás

5.2.3 load.c/h - Fájlbeolvasás és konfiguráció

Felelősség: Adatok betöltése fájlból, konfiguráció kezelése
Főbb függvények:
HRESULT scan(const char *filename, RecordHandler handler, void *data, PROCESS mode);
// Fájl soronkénti feldolgozása állapotgéppel

HRESULT test_and_run_config(const char *config_file, PROCESS mode);
// Konfigurációs fájl ellenőrzése és betöltése

HRESULT generate_config(FILE *configfile, const char *config_file);
// Új konfigurációs fájl generálása

HRESULT handle_config(char *record, void *data, int *error_flags);
// Konfigurációs rekord feldolgozása

HRESULT handle_base(char *record, void *data, int *error_flags);
// Bázis rekord feldolgozása

HRESULT handle_target(char *record, void *data, int *error_flags);
// Célpont rekord feldolgozása

HRESULT handle_ammo_t(char *record, void *data, int *error_flags);
// Löveg rekord feldolgozása

void LogResult(const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res);
// Eredmények naplózása fájlba

Scan állapotgépe:
NORMAL - Alapállapot
LINE_COMMENT - Sor komment feldolgozás
BLOCK_COMMENT - Blokk komment feldolgozás
RECORD - Adatrekord feldolgozás

5.2.4 storage.c/h - Adatszerkezetek

Felelősség: Adatok tárolása és kezelése láncolt listákban
Főbb adatszerkezetek:

```
// Bázis lista
typedef struct Base_List {
   Base_Node* head;
   Base_Node* tail;
} Base_List;

// Célpont lista
typedef struct Target_List {
   Target_Node* head;
   Target_Node* tail;
} Target_List;

// Löveg típus lista
typedef struct Ammo_t_List {
   Ammo_t_Node* head;
   Ammo_t_Node* tail;
} Ammo_t_List;
```

Főbb függvények (minden lista típushoz):

HRESULT init_*_List(...); // Lista inicializálása

_Node add_*(...); // Elem hozzáadása (rendezett)

int delete_*(...); // Elem törlése ID alapján

*_Node popFront_*(...); // Első elem kiszedése

*_Node popBack_*(...); // Utolsó elem kiszedése

void print_*_List(...); // Lista kiíratása

HRESULT free_*_List(...); // Lista felszabadítása

* Find*ByID(...); // Keresés ID alapján

Strázsált lista design:

head és tail dummy node-ok

Valós elemek: head->next ... tail->prev

Előny: egységes beszúrás/törlés, nincs speciális eset

5.2.5 calculations.c/h - Ballisztikai számítások

Felelősség: Geometriai transzformációk, ballisztikai modellezés

Főbb függvények:

BallisticResult compute_fire_solution(Base *b, Target *t, Ammo_t *ammo);

// Teljes ballisztikai számítás lánca

int SuggestBestBaseAndAmmo(Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo);

// Optimális bázis és löveg keresése (teljes keresés)

int SuggestBestBaseAndAmmo_RegionAware(Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo);

// Regionális optimalizált keresés

static BallisticResult solve_ballistic(Vec3 targetENU, double muzzleVel, double BC);

// Ballisztikai egyenlet numerikus megoldása RK4-vel

static Vec3 wgs84_to_ecef(double lat_deg, double lon_deg, double h);

// WGS84 → ECEF koordináta transzformáció

static Vec3 ecef_to_enu(Vec3 p, Vec3 origin, double lat0_deg, double lon0_deg);

// ECEF → ENU koordináta transzformáció

Numerikus integráció (RK4):

```
static void rk4(State *s, double dt, double BC) {
  State k1, k2, k3, k4, tmp;
  derivatives(s, &k1, BC);
  // ... RK4 lépések
}
```

5.2.6 debug.c/h - Tesztelés és hibakeresés

Felelősség: Automatikus tesztelés, debug funkciók

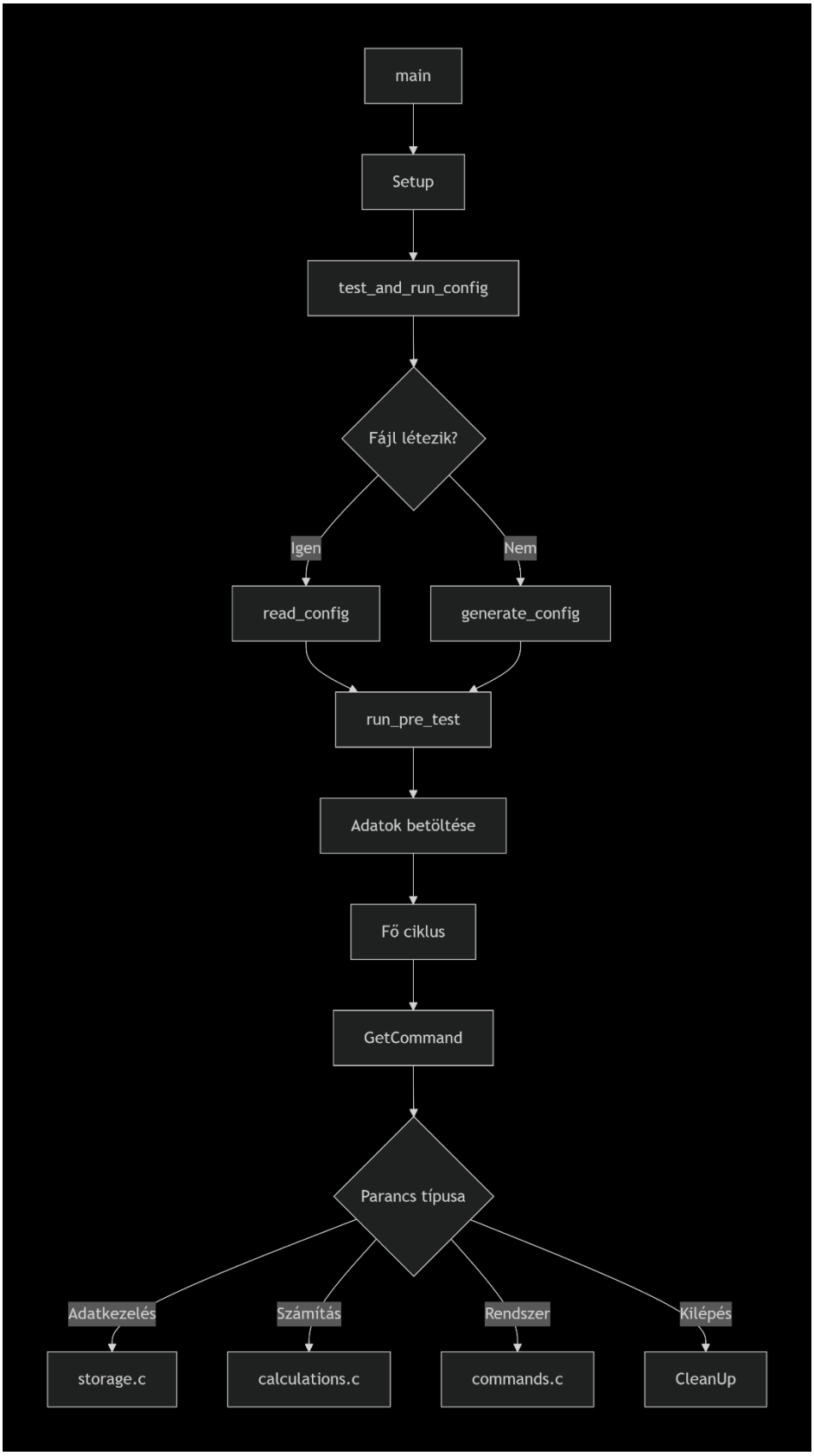
Főbb függvények:

```
HRESULT run_pre_test(PROCESS process_state);
// Automatikus előteszt futtatása

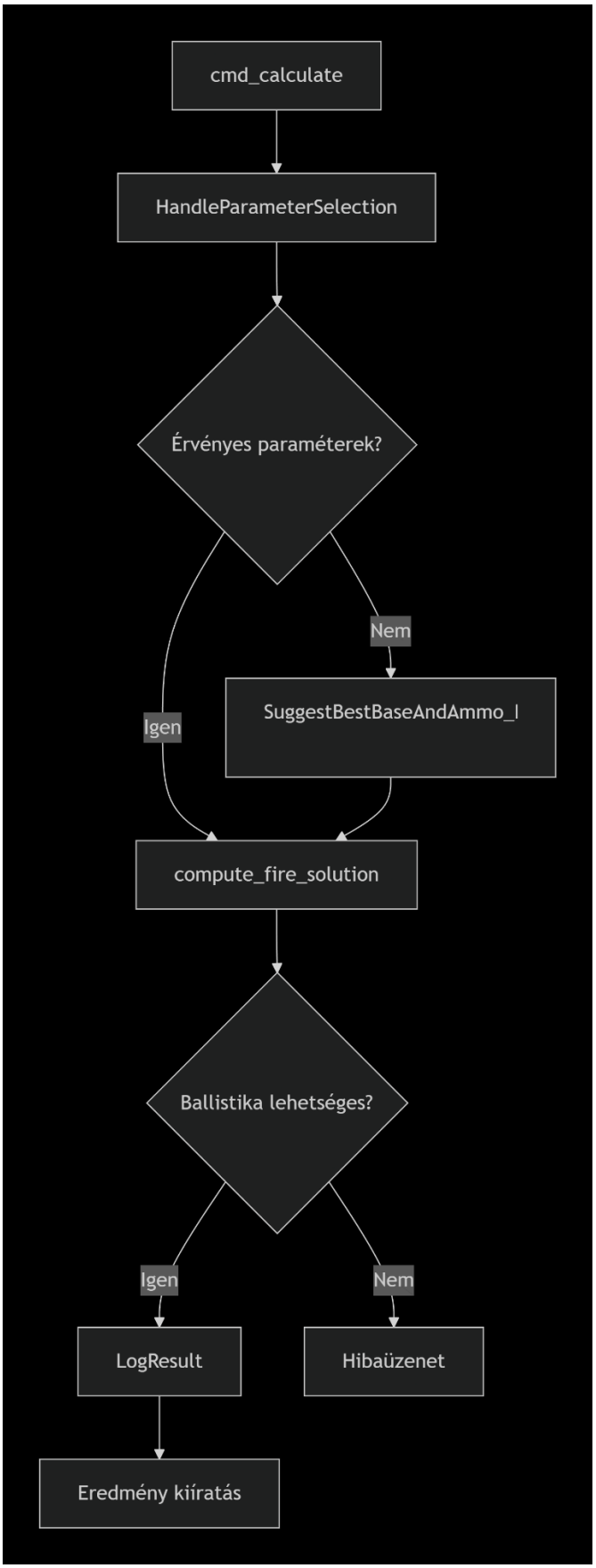
// Színes kimenet függvények (color.h-ben deklarálva):
int print_colored(const char *color_code, const char *text, bool newline);
// Színes szöveg kiíratása
```

5.3 Adatfolyam és vezérlés

5.3.1 Program indítási folyamat



5.3.2 Számítási folyamat



5.4 Memóriakezelés

5.4.1 Dinamikus memória foglalás

```
// Stringek (label, name):
base->label = (char*)malloc(len * sizeof(char));
memcpy(base->label, source, len);

// Struktúrák:
Base* new_base = (Base*)malloc(sizeof(Base));

// Lista node-ok:
Base_Node* new_node = (Base_Node*)malloc(sizeof(Base_Node));
```

5.4.2 Memória felszabadítás

```
// Stringek:
free(base->label);

// Struktúrák:
free(base);

// Lista node-ok:
free(node);

// Teljes lista:
while (list->head->next != list->tail) {
   popBack_List(list);  // Mélységi felszabadítás
}
```

5.4.3 Memóriaszivárgás ellenőrzés

debugmalloc.h használata minden malloc/free művelethez
Valgrind (Linux) vagy Dr. Memory (Windows) kompatibilitás
Program végén teljes memória felszabadítás garantálva

5.5 Hibakezelés

5.5.1 HRESULT hibakódok

#define S_OK 0x00000000 // Sikeres

#define S_FALSE 0x00000001 // Feltételes siker

#define E_FAIL 0x80004005 // Általános hiba

#define E_OUTOFMEMORY 0x8007000E // Memória hiba

#define S_PARTIAL 0x00000002 // Részleges siker

#define S_TERMINATE 0x00000003 // Program leállítás

5.5.2 Hibakezelési stratégia

Előrevetítés: Függvények HRESULT értékkel térnek vissza

Naplózás: Minden hiba részletesen naplózva

Felhasználói visszajelzés: Érthető hibaüzenetek

Erőforrás felszabadítás: Hiba esetén is tisztítás

5.6 Optimalizációk

5.6.1 Regionális keresés

ID alapú regionok: region = id / 1000

Kétfázisú keresés: Először saját region, utána minden más

Korai levágás: Távolság alapú optimalizáció

5.6.2 Lista műveletek

Rendezett beszúrás: O(log n) keresés + O(1) beszúrás

Optimalizált keresés: Irányválasztás a lista eleje/vége távolsága alapján

Strázsák: Egyszerűsített él-kezelés

5.6.3 Numerikus számítások

RK4 integráció: Pontosság/gyorsaság egyensúly

Légellenállás modell: G7 drag coefficiens táblázat

Koordináta transzformációk: Előszámolt trigonometrikus értékek

5.7 Bővíthetőség és karbantarthatóság

5.7.1 Moduláris design

Laza kapcsolat: Modulok függetlenek, jól definiált interfészek

Egyszerű tesztelés: Minden modul külön tesztelhető

Konfigurálhatóság: Külső fájlokkal történő beállítás

5.7.2 Konvenciók és stílus

Elnevezés: konzisztens camelCase/hungarian notation

Kommentelés: Doxygen kompatibilis header kommentek

Hibakezelés: egységes HRESULT rendszer

Memóriakezelés: minden malloc-hoz tartozik free

---

6. Teszt dokumentáció

6.1 Tesztelési stratégia

6.1.1 Tesztelési szintek

Egységteszt: Egyes függvények külön tesztelése

Integrációs teszt: Modulok együttműködésének tesztelése

Rendszerteszt: Teljes program funkcionalitásának ellenőrzése

Teljesítményteszt: Nagy adatmennyiség alatti működés

6.1.2 Automatikus tesztek

run_pre_test() függvény - teljes funkcionalitás tesztelése

Konfiguráció automatikus generálása

Adatbetöltés és feldolgozás validálása

6.2 Tesztesetek

6.2.1 Funkcionális tesztek

1. Konfigurációs teszt

// Teszt: Hiányzó config.txt

// Elvárt: Új konfig generálás, felhasználói interakció

// Tesztfájl: Nincs (első indítás szimuláció)

// Teszt: Érvényes config.txt
// Elvárt: Sikeres betöltés, mappa és színmód beállítás
// Tesztfájl: config.txt (manuálisan létrehozva)

2. Adatbetöltési teszt
// Teszt: Helyes formátumú bázis fájl
// Elvárt: 15 bázis sikeres betöltése
// Tesztfájl: bases.txt (15 érvényes rekord)

// Teszt: Hibás formátumú fájl
// Elvárt: Hibás sorok kihagyása, hibaüzenet
// Tesztfájl: test_invalid.txt (hibás sorok keverve)

// Teszt: Kommentek kezelése
// Elvárt: Komment sorok teljes figyelmen kívül hagyása
// Tesztfájl: Tetszőleges fájl // és /* */ kommentekkel

3. Adatszerkezet teszt
// Teszt: Bázis lista műveletek
// Elvárt: Hozzáadás, törlés, keresés, rendezés helyes működése
// Teszt: add_Base(), delete_Base(), FindBaseByID(), print_Base_List()

// Teszt: Memóriakezelés
// Elvárt: Nincs memóriaszivárgás
// Teszt: Valgrind/drmemory futtatás teljes programon

4. Számítási teszt
// Teszt: Ballisztikai számítás
// Elvárt: Értelmes eredmények ismert bemenetekre
// Teszt: compute_fire_solution() ismert paraméterekkel

// Teszt: Regionális keresés
// Elvárt: Saját region előnyben részesítése
// Teszt: SuggestBestBaseAndAmmo_RegionAware() különböző ID-kkel

// Teszt: Optimális bázis keresés
// Elvárt: Legközelebbi elérhető bázis kiválasztása

// Teszt: Különböző távolságú bázisokkal

5. Parancsértelmező teszt
// Teszt: Érvényes parancsok
// Elvárt: Megfelelő handler meghívása
// Teszt: GetCommand() minden támogatott paranccsal

// Teszt: Érvénytelen parancsok
// Elvárt: Hibaüzenet, help információ
// Teszt: Véletlenszerű karaktersorozatok

// Teszt: Kontextus függő parancsok
// Elvárt: Parancsok csak megfelelő kontextusban elérhetők
// Teszt: Parancsok különböző menükben

6.2.2 Határérték tesztek

1. Nagy adatmennyiség
// Teszt: 1000+ bázis betöltése
// Elvárt: Sikeres betöltés, elfogadható teljesítmény
// Megjegyzés: Generált tesztfájl szükséges

// Teszt: Nagyon távoli célpontok
// Elvárt: "No solution" válasz, nem végtelen ciklus
// Bemenet: 10000+ km távolság

2. Speciális esetek
// Teszt: Duplikált ID-k
// Elvárt: Normalizálás vagy figyelmeztetés
// Bemenet: I0 ...; I0 ...; (azonos ID különböző adatokkal)

// Teszt: Üres fájlok
// Elvárt: Sikeres betöltés (0 elem), nincs crash
// Bemenet: Teljesen üres vagy csak kommenteket tartalmazó fájl

// Teszt: Nagyon kis/nagy koordináták
// Elvárt: Érvényesség ellenőrzés, hibakezelés

// Bemenet: latitude = 100.0 (érvénytelen)

3. Memória és erőforrás
// Teszt: Memória limit közelében
// Elvárt: Graceful failure, nem crash
// Megvalósítás: malloc visszatérési érték ellenőrzése

// Teszt: Fájl I/O hibák
// Elvárt: Hibaüzenet, nem crash
// Teszt: Nem létező fájl, írásvédett fájl, teli lemez

6.3 Tesztadatok

6.3.1 Tesztfájlok

test_bases.txt (15 bázis, különböző regionok)

I0 LSchonherz CLA47.4729283 CLO19.0531429 CA0.0 AMS:{1:100,2:50};
I1 LBudapest CLA47.497912 CLO19.040235 CA96.0 AMS:{1:200,3:75};
I1000 LNewYork CLA40.712776 CLO-74.005974 CA10.0 AMS:{2:150,4:100};
... (további 12 bázis)
test_targets.txt (8 célpont)

I5 LTarget_Alpha CLA48.856613 CLO2.352222 CA35.0 A1.500;
I6 LTarget_Beta CLA52.520008 CLO13.404954 CA34.0 A2.300;
I1005 LTarget_Gamma CLA40.730610 CLO-73.935242 CA10.0 A0.800;
... (további 5 célpont)
test_ammo.txt (42 löveg típus)

I1 N120mm_HE W50.0 D120.0 BR15.0 MV650.0 BC0.300;
I2 N155mm_HE W43.0 D155.0 BR25.0 MV827.0 BC0.350;
I3 N203mm_NUC W110.0 D203.0 BR500.0 MV650.0 BC0.420;
... (további 39 löveg típus)
test_invalid.txt (hibás formátumok)

// Érvénytelen sor - hiányzó pontosvessző
I0 LTest CLA40.0 CLO20.0 CA0.0

// Kevert sorrend
CLO20.0 CA0.0 I1 CLA40.0 LTest2;

// Hiányzó paraméterek
I2 LTest3 CA0.0;

// Érvénytelen számformátum
I3 LTest4 CLAinvalid CLO20.0 CA0.0;

6.3.2 Automatikus tesztgenerálás

A tesztadatok részben generálva vannak szkriptekkel a következő szempontok figyelembevételével:

Különböző regionok (0-999, 1000-1999, stb.)

Különböző távolságok (közeli, közepes, távoli)

Különböző magasságkülönbségek

Speciális esetek (érintkező koordináták, stb.)

6.4 Teszt végrehajtás

6.4.1 Manuális tesztelés

1. Alap funkcionalitás teszt
> test
2. Konkrét parancsok tesztelése
> bases
> list
> add I999 LTest CLA0.0 CLO0.0 CA0.0 AMS:{1:1};
> delete 999
> back
3. Számítási teszt
> calc
> select
> 0 5 1 # ismert működő paraméterek

6.5.2 Teljesítmény metrikák

Tesztkörnyezet:

Hardware:

Processor	12th Gen Intel(R) Core(TM) i5-12450HX 2.40 GHz
Installed RAM	16.0 GB (15.7 GB usable)
Storage	477 GB SSD
Graphics Card	NVIDIA GeForce RTX 2050 (4 GB), Intel(R) UHD Graphics (128 MB)
System Type	64-bit operating system, x64-based processor

OS:

Edition	Windows 10 Home
Version	22H2
Installed on	7/16/2025
OS Build	19045.6466

// Adatbetöltés sebessége:

// - 15 bázis: < 0.1 másodperc

// - 8 célpont: < 0.05 másodperc

// - 42 löveg: < 0.1 másodperc

// Összesen: < 0.25 másodperc

// Számítási sebesség:

// - Egyszerű eset: < 0.5 másodperc

// - Komplex keresés: 1-3 másodperc

// - Maximum (nagy adat): < 300 másodperc

// Memóriahasználat:

// - Alap üres: ~2 MB

// - Teljes adat betöltés után: ~5 MB

// - Maximum terhelés alatt: < 50 MB

6.5.3 Hibák és javítások

Javított hibák:

Memória felszabadítás hiány – javítva: teljes mélységi felszabadítás

Float pontosság problémák – javítva: epszilon érték használata összehasonlításokhoz

Fájl bezárás hiánya – javítva: minden fopen-hoz fclose páros

Ismert korlátozások:

Nagyon nagy adatmennyiség (>10,000 elem) lassú lehet

Windows alatti színes kimenet speciális beállítás kell

Csak WGS84 koordináta rendszer támogatott

Egyszerűsített légellenállás modell